



**Universidad
Europea**

**PROYECTO INTEGRADOR
1º DAW**



PROYECTO INTEGRADOR 1º DAW CURSO 2025-26

Contenido

1. Descripción breve	2
2. Módulos Formativos involucrados	6
3. Objetivos generales del proyecto	6
4. Objetivos específicos del Proyecto (vinculados a capacidades de los módulos).....	6
5. Metodología:.....	8
6. Secuenciación / temporalización del proyecto:.....	8
7. Material necesario:	11
8. Release y defensa del P.I.:	11
9. Instrumentos de evaluación:	11
10. Equipos:	12

1. Descripción breve

APLICACIÓN DE GESTIÓN DE CITAS DEL TALLER DE EDNA MODA:

La afamada diseñadora de vestimenta superheroica Edna Moda nos ha pedido una aplicación de gestión de citas para su taller de sastrería.

La aplicación permitirá un **CRUD** básico:

- Creación de citas de sastrería y de clientes.
- Borrado de citas y de clientes
- Modificación de citas y de clientes
- Consulta y listado de citas y de clientes

La información que vamos a manejar será la siguiente:

- Los **clientes**:
 - Se identifican con un número de cliente único
 - Nombre de superhéroe / supervillano
 - Superpoder
 - Colores
- El **traje** de cada cliente:
 - Identificador
 - Nombre (que indique si es el traje principal del cliente, o uno específico).
 - Estado (en diseño, costura o taller).
- Los **talleres** del Estudio de Moda:
 - Se identifican con un número único
 - Nombre de sala (que son nombres de ciudades punteras en moda, como Milán, París, Madrid...)
 - Tipo de sala (ej.- diseño, costura, pruebas).

- Los **empleados** que trabajan en el taller de la Sra. Moda:
 - Un identificador único
 - Nombre y apellidos del empleado
 - Apodo del empleado (nombre en clave)
 - Categoría:
 - Aprendiz – no atiende a clientes sin un oficial a cargo (no puede crear citas nuevas).
 - Oficial – acceso a todos los talleres
 - Maestro – acceso a todos los talleres, y a clientes (creación, modificación, y borrado).
- La información que se necesita sobre las **citas** es la siguiente:
 - Identificador único
 - Para qué cliente es
 - Para qué traje del cliente es
 - Empleado que le atiende, que puede ser Oficial o Maestro (y aprendices, hasta 2).
 - Taller que se reserva (que ya nos dice el tipo de cita que nos ocupa).
 - Día, Hora y Duración de la cita (en horas completas) (por defecto, 1h).

Los requisitos mínimos para la interfaz deberán ser los siguientes:

- Una pantalla de login, donde los empleados puedan acceder con usuario y contraseña
- La aplicación puede ser usada por cualquier empleado, pero tendrán distintos permisos dependiendo de su categoría.

- Los aprendices pueden ver las citas en las que trabajan, pero no pueden crear nuevas ni modificarlas.
- Los oficiales pueden
 - Ver todas las citas
 - Modificar solo las citas suyas (el resto aparecen bloqueadas)
 - crear citas de diseño, costura y pruebas, y asignar aprendices a las citas. Cada cita puede tener de 0 a 2 aprendices.
- Los maestros pueden
 - Ver y modificar todas las citas que hay
 - reservar citas donde quieran y cuando quieran, y asignarlas al oficial que quieran.
 - añadir, modificar y borrar clientes, y crear, borrar y modificar las citas actuales.
 - Añadir, modificar y borrar talleres disponibles en la sede

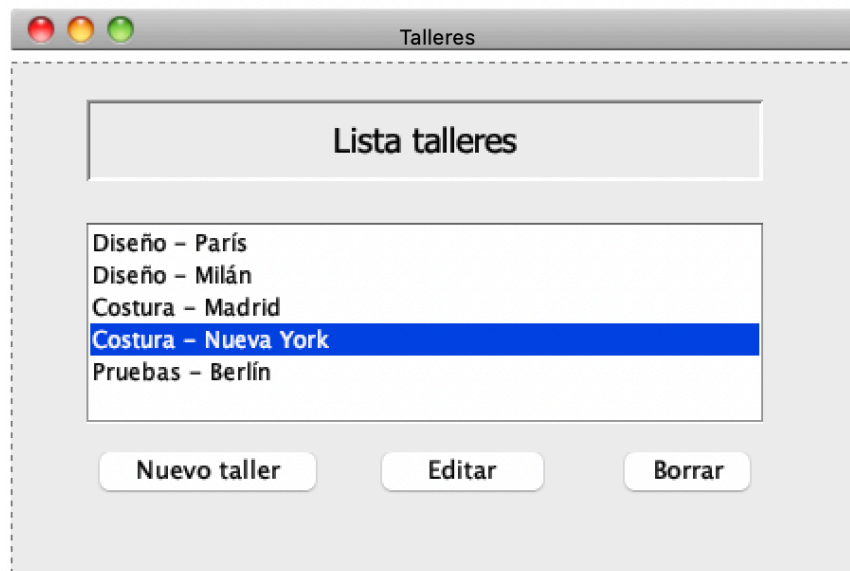
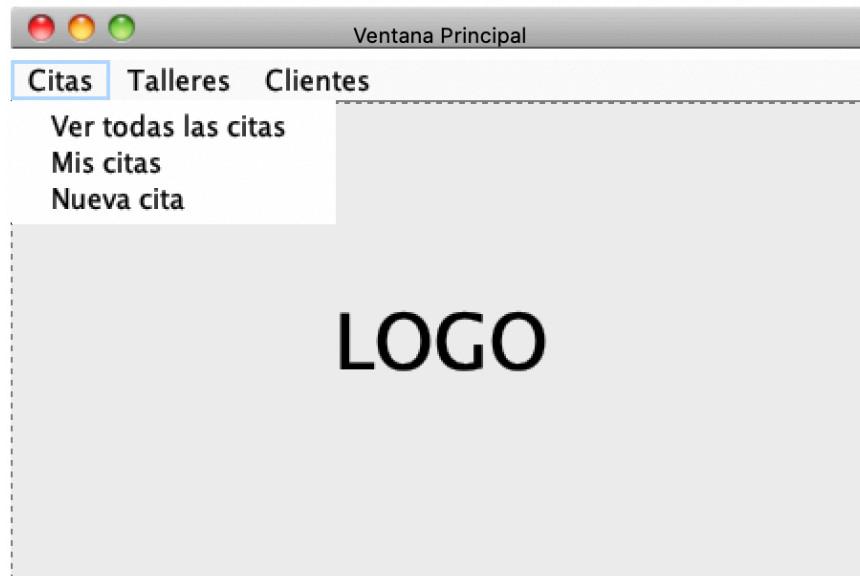
BONUS:

La cliente está dispuesta a pagar más por una funcionalidad específica: guardar si los clientes son héroes o villanos, y actuar en consecuencia. Ella no hace distinciones a la hora de diseñar trajes, y se mueve bien entre grises, aunque sus clientes no tanto.

Esta distinción servirá para poder organizar mejor la agenda:

- **un héroe y un villano no podrán tener citas uno a continuación del otro**, para evitar la lucha a muerte que destrozaría la sede si, por casualidad, se ven por los pasillos.
- Se necesita por lo menos una hora de margen entre dos citas de este tipo.

A continuación, se muestran unos ejemplos de interfaces para que tengáis una idea de a qué nos referimos. **Estas interfaces son solo ejemplos**, y están para que os sirvan de inspiración; por favor, no os baséis en ellas literalmente para la elaboración de vuestra aplicación.



2. Módulos Formativos involucrados

- Programación [PR]
- Bases de Datos [BD]
- Entornos de Desarrollo [ED]

3. Objetivos generales del proyecto

Mediante este trabajo se pretende que los alumnos consigan:

- Gestionar y realizar el trabajo del área asignada, en relación con las cargas de trabajo propias del Ciclo Formativo (planificación del tiempo).
- Realizar consultas a la persona adecuada y saber respetar la autonomía de los compañeros.
- Mantener el espíritu de innovación y actualización en el ámbito de su trabajo, buscando habilidades para la sinergia de equipos y liderazgo.
- Gestionar eficazmente los conflictos que se puedan producir, mediando y contribuyendo al establecimiento de un ambiente de trabajo agradable y actuando de forma respetuosa y tolerante.
- Resolver problemas y tomar decisiones individuales, siguiendo las normas y procedimientos establecidos, definidos dentro del ámbito de su competencia.

4. Objetivos específicos del Proyecto (vinculados a capacidades de los módulos)

Bases de datos

- Crear bases de datos definiendo su estructura y las características de sus elementos según el modelo relacional
- Diseñar modelos lógicos normalizados interpretando diagramas entidad/relación.

- Realizar el diseño físico de bases de datos utilizando asistentes, herramientas gráficas y el lenguaje de definición de datos.
- Consultar y modificar la información almacenada utilizando asistentes, herramientas gráficas y el lenguaje de manipulación de datos.

Programación

- Realizar el diseño de clases necesarias para seguir el patrón MVC.
- Implementar cada una de las clases para lograr su objetivo: clases de interfaz gráfica pertenecientes a la vista, clases que representen el modelo y clases encargadas de la lógica de la aplicación pertenecientes al control.
- Desarrollar una aplicación que gestione información almacenada en bases de datos relacionales identificando y utilizando mecanismos de conexión.

Esta tarea permitirá alcanzar la competencia completa para desarrollar componentes software en lenguajes de programación orientados a objetos.

Entornos de Desarrollo

- Realizar el análisis y el diseño de cualquier aplicación empleando técnicas UML.
- Documentar aplicaciones con JavaDoc.
- Gestionar las diferentes versiones de un software y el trabajo colaborativo.
- Realizar depuración y testeo sobre los programas.
- Seguir metodologías ágiles para el desarrollo y planificación de programas.

5. Metodología:

Para el proyecto se utilizará la **metodología ágil SCRUM**, un framework de desarrollo ágil de software para gestionar el desarrollo de productos.

- Se define como una estrategia flexible e integral de desarrollo de producto donde un equipo trabaja como una unidad para alcanzar un objetivo: los retos asumen un enfoque secuencial del desarrollo de producto, y se habilita a los equipos para organizarse por sí mismos, animando a la colaboración presencial u online de todos los miembros del equipo, así como a una comunicación diaria cara a cara de todos los miembros y perfiles en el proyecto.
- Una idea clave de SCRUM es el reconocimiento de que, durante el desarrollo del proyecto, el cliente puede cambiar de idea sobre lo que quiere o necesita, y que puede haber cambios imprevistos no especificados en la planificación original.
- SCRUM adopta una aproximación práctica que plantea que el problema no se puede entender de forma integral desde un principio, y que supone un desafío y una oportunidad para la capacidad de los miembros del equipo de desarrollo el adaptarse al cambio y encontrar soluciones a los nuevos problemas.

6. Secuenciación / temporalización del proyecto:

El proyecto se desarrollará a lo largo del tercer trimestre.

Al final de cada sprint se realizará una presentación del proyecto junto a los objetivos conseguidos (reunión de revisión + retrospectiva). En éste se identificarán oportunidades de mejora en el DT (Development Team), comunicación, prácticas, etc.

Además, al final de cada sprint se establece la línea de desarrollo (Branch) del proyecto a continuar tras las presentaciones de los diferentes equipos. Así se pretende reproducir la dinámica de trabajo de una empresa de desarrollo donde existe sinergia entre los equipos de desarrollo y la aplicación que crece con las diferentes líneas de trabajo.

Los sprints tendrán una duración de **2 semanas**, aproximadamente.

Sprint #1: 3 de marzo – 16 marzo (2 semanas)

[PR1] Presentación del módulo.

[BD1] Presentación del módulo.

[ED1] Presentación del módulo.

[PR2] Creación del proyecto.

Planificación detallada de los requisitos de la aplicación.

[ED2] Análisis de las especificaciones del proyecto.

[ED3] Proyecto en GitHub. Planificación con Trello.

[BD2] Análisis de las especificaciones del proyecto. Diagrama E/R.

[ED4] Apartados de la memoria: Portada, Índice, Objetivos.

Sprint Review #1 (17 marzo)

Sprint #2: 17 de marzo – 30 de marzo (Semana Santa)

[PR3] Diseño de la interfaz de las ventanas.

Creación de las clases pertenecientes a la Vista.

[BD3] Generación del Modelo Relacional.

Normalización

[BD4] Creación de la BBDD: tablas, índices, etc.

[BD5] Inserción de los datos necesarios para la aplicación.

[ED5] Diagrama de casos de uso.

[ED6] Diseño del logo.

[ED7] Apartados de la memoria: Tecnologías utilizadas, Metodologías,
Anexo 1 – Listado de requisitos de la aplicación.

Sprint Review #2 (7 de abril)

Sprint #3: 7 de abril – 20 de abril (2 semanas)

[ED8] Generación del diagrama de clases.

[PR4] A partir del diagrama de clases:

- Creación de las clases pertenecientes al Modelo
- Creación de las clases pertenecientes al Control encargadas del acceso a BBDD y del manejo de la aplicación.

[ED9] Apartados de la memoria: Introducción, Desarrollo.

Sprint Review #3 (21 abril)

Sprint #4: 21 de abril – 4 mayo (2 semanas)

[PR5] Refinamiento del programa.

[ED10] Realización de pruebas JUnit.

Documentación de la aplicación JavaDoc.

[PR6] [BD6] Integración de todos los elementos y solución de las incidencias de integración que se produzcan.

[ED11] Realización del manual de usuario en la wiki de GitHub usando Markdown.

[ED12] Memoria completa, excepto el Anexo II con las capturas de la aplicación.

Project Review (5 de mayo)

Preparación release: 5 de mayo – 12 mayo

Elaboración de la presentación.

Revisión de la documentación, tanto JavaDoc como manual de usuario y wiki de GitHub usando Markdown.

Release: martes 12 mayo 2026

7. Material necesario:

- JDK de Java actualizado.
- Eclipse IDE
- Software Ideas Modeler, draw.io o equivalente
- Software de BBDD relacionales: MySQL o equivalente

8. Release y defensa del P.I.:

- Instalar en el aula la aplicación para mostrar su funcionamiento.
- Memoria del proyecto.
- Presentación del proyecto (exposición y defensa).
- Fecha: 12 de mayo de 2026.

9. Instrumentos de evaluación:

1. Si el proyecto tiene errores de compilación se considerará **no apto**.
2. Se pondrá especial énfasis en valorar positivamente la **ejecución** (funcionamiento, adecuación a las especificaciones, estabilidad, etc.) y el **diseño** (documentación, GitHub, etc.).
3. Presentación y evaluación de los objetivos en cada Sprint Review
4. También se evaluarán las competencias actitudinales y de trabajo colaborativo de forma individual, así como la actividad en Trello y GitHub.

Finalmente se calculará la nota teniendo en cuenta la siguiente ponderación:

- 70% Competencias técnicas
- 20% Habilidades para el trabajo en equipo
- 10% Actitud individual y participación en la tarea.

10. Equipos:

Se establecerán los equipos durante la presentación del proyecto.

Habrá **3-4 personas** por equipo.